
Guía de referencia · Data Science & ML

Analítica Avanzada y Modelado de Datos

KDD · Aprendizaje Supervisado · No Supervisado · Semisupervisado

Validación · Bias-Varianza · Grid Search · Métricas de Evaluación

Índice

1. Extracción del Conocimiento: el Proceso KDD

1.1. Fases del proceso KDD

2. Tipos de Problemas de Machine Learning

2.1. Aprendizaje Supervisado

2.2. Aprendizaje No Supervisado

2.3. Aprendizaje Semisupervisado

3. Inferencia vs Predicción

4. Scikit-learn y el Ecosistema Python para ML

5. Entrenamiento y Validación de Modelos

5.1. Enfoques de partición

5.2. Desbalanceo de clases

6. Overfitting, Underfitting, Bias y Varianza

6.1. Overfitting (Sobreaajuste)

6.2. Underfitting (Infraajuste)

6.3. Bias (Sesgo) — Error Sistemático

6.4. Variance (Varianza)

6.5. Resumen: el trade-off Bias-Varianza

7. LOOCV y Ajuste de Hiperparámetros

7.1. LOOCV (Leave-One-Out Cross Validation)

7.2. Grid Search: Ajuste de Hiperparámetros

8. Métricas de Evaluación

8.1. Distinción clave: Validación vs Evaluación

8.2. Métricas para Clasificación

— Matriz de Confusión

— Métricas derivadas

— Curva ROC y AUC

8.3. Métricas para Regresión

8.4. Métricas para Clustering (No Supervisado)

— Método del Codo (Elbow Method)

— Coeficiente de Silueta

1. Extracción del Conocimiento: el Proceso KDD

El **KDD** (*Knowledge Discovery in Databases*) es el proceso completo de extracción de conocimiento útil, válido y comprensible a partir de datos. No es simplemente aplicar un modelo: es un flujo metodológico end-to-end.

1.1. Fases del proceso KDD

Fase	Qué se hace
1. Selección	Elegir variables, períodos, segmentos o áreas geográficas relevantes para el problema
2. Preprocesamiento	Limpieza de datos, tratamiento de nulos, eliminación de ruido e inconsistencias
3. Transformación	Feature engineering, normalización/estandarización, reducción de dimensionalidad
4. Modelización	Aplicación de algoritmos: regresión, clasificación, clustering, reglas de asociación
5. Interpretación	Analizar resultados, validar que tienen sentido de negocio y traducirlos en conocimiento

■ El proceso es iterativo, no lineal. Los resultados de la fase 5 frecuentemente obligan a volver a fases anteriores para ajustar la selección de variables o el preprocesamiento.

2. Tipos de Problemas de Machine Learning

Los problemas de ML se clasifican según si se dispone o no de etiquetas en los datos de entrenamiento.

2.1. Aprendizaje Supervisado

El algoritmo aprende a partir de un conjunto de datos **etiquetado**: se conoce **y** para cada observación y el objetivo es aprender la relación $X \rightarrow y$ para predecir sobre nuevos datos.

- **Regresión** — variable objetivo numérica continua (precio, temperatura, volumen de ventas).
- **Clasificación** — variable objetivo categórica (¿comprará?, ¿es fraude?, ¿es spam?).

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LinearRegression

# Clasificación
clf = RandomForestClassifier(random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

# Regresión
reg = LinearRegression()
reg.fit(X_train, y_train)
y_pred = reg.predict(X_test)

```

■ Patrón universal de scikit-learn: `fit(X_train, y_train)` para entrenar, `predict(X_test)` para inferir. Todos los modelos comparten esta interfaz.

2.2. Aprendizaje No Supervisado

No existe variable objetivo. El objetivo es **descubrir la estructura subyacente** de los datos: agrupaciones, patrones ocultos o simplificaciones no visibles a simple vista.

- **Clustering** — agrupa observaciones similares entre sí. Algoritmo más usado: K-Means.
- **Reglas de asociación** — detecta relaciones "si ocurre A, suele ocurrir B". Algoritmo: Apriori.
- **Reducción de dimensionalidad** — sintetiza muchas variables en pocas sin perder información crítica. Algoritmo más usado: PCA.

```

from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# Clustering
kmeans = KMeans(n_clusters=4, random_state=42)
df['cluster'] = kmeans.fit_predict(X)

# Reducción de dimensionalidad
pca = PCA(n_components=2)
X_reducido = pca.fit_transform(X)

```

2.3. Aprendizaje Semisupervisado

Combina supervisado y no supervisado cuando se dispone de **pocos datos etiquetados** y muchos sin etiquetar. Etiquetar manualmente toda la población es difícil, costoso o imposible.

Técnica	Descripción
Self-Training	Entrena con etiquetados, predice los no etiquetados, incorpora predicciones con alta confianza (p. ej. >
Cluster-then-label	Clusteriza toda la población y asigna la etiqueta mayoritaria del cluster a miembros no etiquetados.

Técnica	Descripción
Label Propagation	Conecta los datos por una red de similitud; las etiquetas conocidas se propagan hacia nodos vecinos

```
from sklearn.semi_supervised import LabelPropagation
import numpy as np

# -1 indica observación NO etiquetada
y_semi = np.copy(y_train)
y_semi[indices_no_etiquetados] = -1

lp = LabelPropagation()
lp.fit(X_train, y_semi)
y_pred = lp.predict(X_test)
```

3. Inferencia vs Predicción

Dos objetivos distintos que determinan qué modelo usar y cómo interpretarlo.

	Inferencia	Predicción
Pregunta	¿Cómo y cuánto influye cada variable?	¿Cuál será el valor con el menor error posible?
Prioridad	Interpretabilidad y causalidad	Exactitud
Modelos	Regresión lineal/logística (coeficientes interpretables)	Boosting, redes neuronales (caja negra)
Casos de uso	Análisis de sensibilidad, políticas de negocio	Logística, detección de fraude, bolsa

Inferencia

El modelo cuantifica el efecto de cada variable. Un coeficiente β negativo en una regresión de ventas indica que subir el precio las reduce, y cuantifica ese impacto.

```
import statsmodels.api as sm

X_con_cte = sm.add_constant(X_train)
modelo = sm.OLS(y_train, X_con_cte).fit()
print(modelo.summary()) # coeficientes, p-valores, intervalos de confianza
```

Predicción

El modelo maximiza la exactitud sin necesidad de ser interpretable.

```
from sklearn.ensemble import GradientBoostingRegressor

modelo = GradientBoostingRegressor(n_estimators=200, random_state=42)
modelo.fit(X_train, y_train)
y_pred = modelo.predict(X_test)
```

4. Scikit-learn y el Ecosistema Python para ML

Scikit-learn es la librería de referencia para ML tradicional en Python. Open source, construida sobre NumPy, SciPy y Matplotlib. Cubre el ciclo completo: preprocesamiento, modelado, evaluación y selección de modelos.

Librería	Cuándo usarla
Scikit-learn	ML tradicional: clasificación, regresión, clustering, reducción de dimensionalidad
XGBoost / LightGBM / CatBoost	Máxima precisión en datos tabulares; competiciones o entornos de alta exigencia
PyTorch / TensorFlow-Keras	Deep Learning, datos no estructurados (imágenes, texto, audio), arquitecturas avanzadas
Statsmodels	Inferencia estadística pura, contraste de hipótesis, análisis econométrico
Prophet	Forecasting de series temporales con estacionalidad y efectos de calendario

5. Entrenamiento y Validación de Modelos

En ciencia de datos se subdivide el conjunto de datos para garantizar que el modelo **generaliza** correctamente, no solo memoriza los datos de entrenamiento.

5.1. Enfoques de partición

Clásico — Train/Test (80/20)

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Riguroso — Train/Validation/Test (70/15/15)

```
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)
```

Robusto — Validación Cruzada (K-Fold)

Una única partición 80/20 puede estar sesgada por azar. K-Fold repite el proceso K veces rotando el bloque de test. El rendimiento final es el promedio de las K iteraciones.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(modelo, X, y, cv=5, scoring='neg_mean_squared_error')
print(f"RMSE medio: {(-scores.mean())**0.5:.4f}")
```

5.2. Datos no representativos: desbalanceo de clases

Si la variable objetivo está desequilibrada, un muestreo aleatorio simple producirá un modelo sesgado. La solución es **balancear** antes de entrenar.

```
from imblearn.over_sampling import SMOTE

sm = SMOTE(random_state=42)
X_bal, y_bal = sm.fit_resample(X_train, y_train)
```

Técnica	Estrategia
SMOTE	Genera muestras sintéticas interpolando entre casos minoritarios
ADASYN	Igual que SMOTE pero focaliza la generación en zonas difíciles de clasificar
Random Oversampling	Duplica muestras minoritarias aleatoriamente

6. Overfitting, Underfitting, Bias y Varianza

6.1. Overfitting (Sobreajuste)

El modelo aprende los datos de entrenamiento **demasiado bien**, incluyendo su ruido.

Aspecto	Detalle
Causa	Modelo excesivamente complejo para el volumen de datos disponible
Síntoma	Error muy bajo en train (~10%) y alto en test (~40%)
Indicador clave	Brecha grande entre métricas de train y test

Aspecto	Detalle
Solución	Regularización, más datos, reducir complejidad del modelo

6.2. Underfitting (Infraajuste)

El modelo es **demasiado simple** para capturar la estructura de los datos.

Aspecto	Detalle
Causa	Modelo sub-parametrizado, variables relevantes omitidas
Síntoma	Error elevado tanto en train como en test
Indicador clave	Ambas métricas son malas por igual
Solución	Aumentar complejidad del modelo, añadir variables relevantes

6.3. Bias (Sesgo) — Error Sistemático

El modelo falla **de la misma forma** independientemente de los datos.

Aspecto	Detalle
Causa	Suposición errónea sobre la distribución, modelo demasiado rígido
Síntoma	Distancia sistemática entre predicción media y valor real
Ejemplo	Aproximar una curva de puntos con una línea recta

6.4. Variance (Varianza) — Sensibilidad al Ruido

El modelo cambia sus estimaciones drásticamente ante pequeñas variaciones en los datos de entrada.

Aspecto	Detalle
Causa	Sobre-parametrización, datasets pequeños con modelos muy complejos, outliers
Síntoma	Excelente en train, pésimo en test
Ejemplo	Polinomio de grado 20 o red neuronal profunda sin regularización

6.5. Resumen: el trade-off Bias-Varianza

	Bias alto	Bias bajo
Varianza alta	Peor escenario posible	Overfitting
Varianza baja	Underfitting	Modelo ideal ✓

```

from sklearn.model_selection import learning_curve
import numpy as np

train_sizes, train_scores, val_scores = learning_curve(
    modelo, X, y, cv=5, scoring='accuracy',
    train_sizes=np.linspace(0.1, 1.0, 10)
)
# Si train >> val -> overfitting (alta varianza)
# Si ambos bajos -> underfitting (alto sesgo)

```

7. LOOCV y Ajuste de Hiperparámetros

7.1. LOOCV (Leave-One-Out Cross Validation)

Caso extremo de K-Fold donde $K = N$ (número de filas del dataset). Cada iteración entrena con $N-1$ filas y valida con 1. Útil cuando el dataset es muy pequeño.

Aspecto	Detalle
Cuándo usarlo	Datasets muy pequeños (bioestadística, lanzamiento de nuevos productos)
Ventaja	Aprovecha casi todas las muestras para entrenar
Desventaja	Computacionalmente muy costoso; alta varianza si el modelo es sensible

```

from sklearn.model_selection import LeaveOneOut, cross_val_score

loo = LeaveOneOut()
scores = cross_val_score(modelo, X, y, cv=loo, scoring='neg_mean_absolute_error')
print(f"MAE medio: {-scores.mean():.4f}")

```

7.2. Grid Search: Ajuste de Hiperparámetros

Técnica de fuerza bruta sistemática para encontrar la combinación óptima de hiperparámetros según una métrica objetivo. Se combina con K-Fold para que los resultados sean fiables.

```

from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeRegressor

param_grid = {
    'max_depth': [2, 4, 6, 8, 10],
    'min_samples_split': [2, 5, 10]
}

grid = GridSearchCV(
    DecisionTreeRegressor(),
    param_grid,
    cv=5,                      # K-Fold interno
    scoring='neg_mean_squared_error'
)
grid.fit(X_train, y_train)

print(grid.best_params_)      # {'max_depth': 6, 'min_samples_split': 5}
print(grid.best_score_)

```

■ Si defines 2 hiperparámetros con 5 valores cada uno -> 25 modelos x K
 folds = 125 entrenamientos. Con modelos complejos, usar
 RandomizedSearchCV es más eficiente.

8. Métricas de Evaluación

8.1. Distinción clave: Validación vs Evaluación

- **Validación:** medir el comportamiento del modelo durante el entrenamiento para ajustar hiperparámetros y seleccionar el mejor modelo.
- **Evaluación:** medición final del rendimiento sobre el **test set**, que nunca ha sido visto durante el entrenamiento ni la validación.

8.2. Métricas para Clasificación

Matriz de Confusión

Tabla que compara predicciones vs realidad, desglosando aciertos y tipos de error.

```

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm).plot()

```

Métricas derivadas

Métrica	Fórmula	Cuándo priorizarla
Accuracy	$(TP + TN) / \text{Total}$	Clases balanceadas
Precision	$TP / (TP + FP)$	Minimizar falsos positivos (ej. spam)
Recall	$TP / (TP + FN)$	Minimizar falsos negativos (ej. diagnóstico médico)
F1-Score	$2 \cdot (P \cdot R) / (P + R)$	Equilibrio entre Precision y Recall

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Curva ROC y AUC

La curva ROC representa el rendimiento del modelo en **todos los umbrales de decisión posibles**, comparando la tasa de Verdaderos Positivos vs la tasa de Falsos Positivos. El **AUC** resume esto en un único número entre 0 y 1.

■ El umbral óptimo no es siempre 0.5 — depende del coste de cada tipo de error en el contexto de negocio.

```
from sklearn.metrics import roc_auc_score, roc_curve
import matplotlib.pyplot as plt

fpr, tpr, thresholds = roc_curve(y_test, y_prob[:, 1])
auc = roc_auc_score(y_test, y_prob[:, 1])

plt.plot(fpr, tpr, label=f'AUC = {auc:.2f}')
plt.plot([0,1],[0,1], '--')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
plt.legend()
plt.show()
```

8.3. Métricas para Regresión

Todas se basan en medir la distancia entre el valor real y el valor predicho.

Métrica	Fórmula conceptual	Característica
MAE	$\text{mean}(y - \hat{y})$	Fácil de interpretar, robusto a outliers
MSE	$\text{mean}((y - \hat{y})^2)$	Penaliza errores grandes
RMSE	$\text{raiz}(\text{MSE})$	Error en la unidad original de la variable
MAPE	$\text{mean}(y - \hat{y} / y) \cdot 100$	Interpretable como porcentaje

Métrica	Fórmula conceptual	Característica
R^2	1 - SCE/SCT	Proporción de varianza explicada (0–1)

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100

print(f"MAE: {mae:.2f} | RMSE: {rmse:.2f} | R2: {r2:.4f} | MAPE: {mape:.1f}%")
```

■ MAPE explota si hay valores reales igual a 0 (división por cero). En ese caso usar MAE o RMSE.

8.4. Métricas para Clustering (No Supervisado)

Método del Codo (Elbow Method)

Para cada valor de K, calcula el **WCSS** (Within-Cluster Sum of Squares). Se elige el K donde añadir un clúster más ya no reduce el WCSS de forma significativa.

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

wcss = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X)
    wcss.append(km.inertia_)

plt.plot(range(1, 11), wcss, marker='o')
plt.xlabel('Numero de clusteres (K)')
plt.ylabel('WCSS')
plt.title('Metodo del Codo')
plt.show()
```

Coeficiente de Silueta

Mide simultáneamente la **cohesión interna** de cada cluster y la **separación** respecto al resto. Acotado entre -1 y 1.

Valor	Interpretación
≈ 1	El individuo está bien asignado: cerca de los suyos, lejos de los demás
≈ 0	El individuo está en la frontera entre dos clusters

Valor	Interpretación
≈ -1	El individuo se parece más al cluster vecino que al suyo — mala asignación

```
from sklearn.metrics import silhouette_score

score = silhouette_score(X, labels)
print(f"Coeficiente de Silueta: {score:.4f}")
```